

## 第 9 週：記憶體設計

---

本週一句話：FPGA 裡有專用的記憶體硬體，但你要「寫對寫法」它才會用。

### 本週指令

---

雙擊 `env.bat` 開好環境（提示字元要有 `[OSS CAD Suite]`），然後：

```
cd week09_memory
```

每個範例都是這三條（外加一條檢查），把 `04_fifo` 換成你要跑的名字：

```
iverilog -o sim.out 04_fifo.v 04_fifo_tb.v
echo %ERRORLEVEL%
vvp sim.out
gtkwave 04_fifo.gtkw
```

`echo %ERRORLEVEL%` 印 0 才是編譯成功——失敗時 iverilog 常常一個字都不印。

本週範例名稱：

- `01_rom`
- `02_ram_sp`
- `03_ram_dp`
- `04_fifo`

下面每個範例的段落，都直接附了它自己的四行指令。

---

### 學完你要會什麼

---

- [] 會宣告記憶體陣列 `reg [7:0] mem [0:255];`

- [] ★ 知道同步讀取和非同步讀取差在哪，以及為什麼要用同步
- [] 會寫單埠 / 雙埠 RAM
- [] ★★ 會寫 FIFO，說得出空 / 滿怎麼判斷

---

## 一、記憶體怎麼宣告

---

```
reg [7:0] mem [0:255];  
//      ^^^      ^^^  
//      每格多寬      有幾格
```

讀寫：

```
mem[addr] <= din;          // 寫  
dout      <= mem[addr];    // 讀
```

⚠ 注意兩組中括號的意義完全不同：

- [7:0] 在名字前面 → 每一格的寬度
- [0:255] 在名字後面 → 有幾格

---

## 二、★ 同步讀取 vs 非同步讀取

---

```
// 非同步：位址一變資料立刻出來  
assign dout = mem[addr];  
  
// 同步：等一個時脈才出來  
always @(posedge clk) dout <= mem[addr];
```

|            | 非同步         | 同步                 |
|------------|-------------|--------------------|
| 延遲         | 0           | 1 個時脈              |
| FPGA 用什麼實作 | LUT ( 超浪費 ) | Block RAM ( 專用硬體 ) |
| 能做多大       | 幾十位元組就爆     | 幾十 KB 輕鬆           |

[!IMPORTANT] FPGA 裡的 Block RAM 硬體上就是同步的。你寫成非同步，綜合工具只能改用 LUT 拼出來——一個 4Kb 的 BRAM 換成 LUT 要吃掉幾百個邏輯單元。

規則：容量大於幾十位元組，一律寫成同步讀取。代價是所有讀取都晚一拍，設計時要把這一拍算進去。

### 三、單埠 vs 雙埠

|           | 單埠    | 雙埠             |
|-----------|-------|----------------|
| 位址埠       | 1 組   | 2 組            |
| 同時讀 A 寫 B | ✗ 做不到 | ✓ 可以           |
| 佔用資源      | 少     | 多              |
| 典型用途      | 一般暫存  | FIFO、影像緩衝、跨時脈域 |

[!WARNING] 同位址衝突：雙埠 RAM 如果同一拍 `a_addr == b_addr` 而且在寫，B 埠讀到新值還是舊值取決於硬體。不同 FPGA 行為不一樣。設計時不要依賴這個行為。

### 四、★★ FIFO：空 / 滿怎麼判斷

這是 FIFO 唯一的難點，也是面試常考題。

問題：寫指標追上讀指標 → 滿；讀指標追上寫指標 → 空。但兩種情況下 `wr_ptr == rd_ptr`，分不出來。

解法：指標多用一個位元 (wrap bit)

```

reg [AW:0] wr_ptr, rd_ptr;          // ★ 比位址多一位元

assign empty = (wr_ptr == rd_ptr);    // 完全相同 → 空
assign full  = (wr_ptr[AW-1:0] == rd_ptr[AW-1:0]) && // 位址相同
               (wr_ptr[AW]      != rd_ptr[AW]);    // 但繞圈數不同 → 滿
assign count = wr_ptr - rd_ptr;        // 減法自動處理繞回

```

直覺：多的那一位記錄「繞了幾圈」。

- 位址一樣、圈數一樣 → 同一個位置 → 空
- 位址一樣、圈數差一 → 寫的人整整多繞一圈 → 滿

## 五、本週四個範例

```
cd week09_memory
```

### 📁 01\_rom — ROM 與兩種讀取方式 ★

```

iverilog -o sim.out 01_rom.v 01_rom_tb.v
echo %ERRORLEVEL%
vvp sim.out
gtkwave 01_rom.gtkw

```

存的是 0~15 的平方。波形重點：

- `data_async` 跟 `addr` 同時變
- `data_sync` 比 `addr` 晚一個時脈

把兩條線上下排在一起，差別一眼就看到。

### 📁 02\_ram\_sp — 單埠 RAM

```

iverilog -o sim.out 02_ram_sp.v 02_ram_sp_tb.v
echo %ERRORLEVEL%
vvp sim.out
gtkwave 02_ram_sp.gtkw

```

寫 16 格、讀回來比對，再亂數讀寫 300 次全自動驗證。

## 03\_ram\_dp — 雙埠 RAM ★

```
iverilog -o sim.out 03_ram_dp.v 03_ram_dp_tb.v
echo %ERRORLEVEL%
vvp sim.out
gtkwave 03_ram_dp.gtkw
```

關鍵示範：同一拍，A 埠寫位址 `i`，B 埠讀位址 `15-i`。

| 拍 | A埠寫                           | B埠讀                           |
|---|-------------------------------|-------------------------------|
| 0 | <code>mem[ 0] &lt;= f0</code> | <code>mem[15] =&gt; 1f</code> |
| 1 | <code>mem[ 1] &lt;= f1</code> | <code>mem[14] =&gt; 1e</code> |

兩邊同時進行、互不干擾——這是單埠做不到的。

## 04\_fifo — FIFO ★★ 本週重點

```
iverilog -o sim.out 04_fifo.v 04_fifo_tb.v
echo %ERRORLEVEL%
vvp sim.out
gtkwave 04_fifo.gtkw
```

實測：

|      |   |   |   |             |
|------|---|---|---|-------------|
| 寫 a7 | 8 | 0 | 0 |             |
| 寫 a8 | 8 | 0 | 1 | <- 滿了，寫不進去  |
| 讀 a0 | 7 | 0 | 0 | 順序正確        |
| ...  |   |   |   |             |
| 讀 a7 | 0 | 1 | 0 | 順序正確        |
| 讀 -- | 0 | 1 | 0 | <- 空了，讀不出東西 |

同時 寫 51 / 讀 50      count=1      <- 邊寫邊讀，深度維持不變

波形重點：`count` 這條線的起伏，以及 `full` / `empty` 各在哪一拍跳起來。

## 六、練習題

### 練習 1

把 `01_rom` 的內容改成「0~15 的立方」，重跑確認。

## 練習 2 ★

幫 04\_fifo 加上兩個輸出：

- `almost_full`：剩 2 格以下時拉高
- `almost_empty`：只剩 2 筆以下時拉高

( 實務上這兩個訊號很重要 —— 通知上游「快滿了慢一點」 )

## 練習 3 ★★

用 03\_ram\_dp 的雙埠 RAM 重寫 FIFO，讓寫入端和讀出端各用一個時脈 ( 非同步 FIFO )。

提示：指標要用格雷碼 ( 第 4 週練習 2 )，而且跨時脈域時要過雙 FF 同步器 ( 第 7 週 )。

這題很難，是業界真正會用到的東西。做不出來也沒關係，但要理解「為什麼指標要用格雷碼」——因為二進位計數器跨時脈域取樣時，多個位元同時翻轉會取到亂七八糟的中間值，格雷碼一次只變一位元，最多只會差 1。

## 練習 4 ( 思考 )

下面這個 FIFO 的空滿判斷有什麼問題？

```
reg [AW-1:0] wr_ptr, rd_ptr;      // 沒有多留位元
reg [AW:0]    count;

assign empty = (count == 0);
assign full  = (count == 2**AW);
```

► 看答案

## 七、本週檢核

- [ ] 四個範例都跑過
- [ ] 01 的波形上指得出 `data_sync` 比 `data_async` 晚一拍
- [ ] 說得出為什麼 FPGA 要用同步讀取

- [] 不看筆記能解釋 FIFO 的 wrap bit 技巧
  - [] 練習 2 做出來
  - [] 練習 4 答對
- 

## 附註：關於 `sim` 這個指令

---

你可能在別的地方看過我寫的 `sim` 範例名稱。那是我自己做的批次檔（`sim.bat`），不是 Verilog 或 iverilog 的標準指令，教科書上查不到。

它做的事就是把上面那四行包起來、順便幫你檢查結束碼。用不用都可以，也可以刪掉。這份 README 裡給的全部都是原始指令。